

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import plot_tree
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.tree import export_text
import warnings
warnings.filterwarnings("ignore")
```

```
In [2]: df = pd.read_csv("C:\\Users\\sd\\OneDrive\\Desktop\\diabetes.csv")
```

```
In [3]: df
```

```
Out[3]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunc
0	6	148	72	35	0	33.6	0.
1	1	85	66	29	0	26.6	0.
2	8	183	64	0	0	23.3	0.
3	1	89	66	23	94	28.1	0.
4	0	137	40	35	168	43.1	2.
...	...	...	...	...	...	...	...
763	10	101	76	48	180	32.9	0.
764	2	122	70	27	0	36.8	0.
765	5	121	72	23	112	26.2	0.
766	1	126	60	0	0	30.1	0.
767	1	93	70	31	0	30.4	0.

768 rows × 9 columns



This dataset may contain missing values represented by zeros in certain columns like Glucose, BloodPressure, SkinThickness, Insulin, and BMI. It's important to identify and handle these missing values appropriately as they can significantly impact model performance and health predictions. For instance, you might replace zero values in these columns with the mean or median of the non-zero values in the respective columns.

Data processing

In [4]: `print(df.isnull().sum())`

```
Pregnancies      0
Glucose           0
BloodPressure     0
SkinThickness     0
Insulin           0
BMI               0
DiabetesPedigreeFunction  0
Age               0
Outcome           0
dtype: int64
```

In [5]: `df.describe()`

Out[5]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Diabet
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	

In [6]: `df.count()`

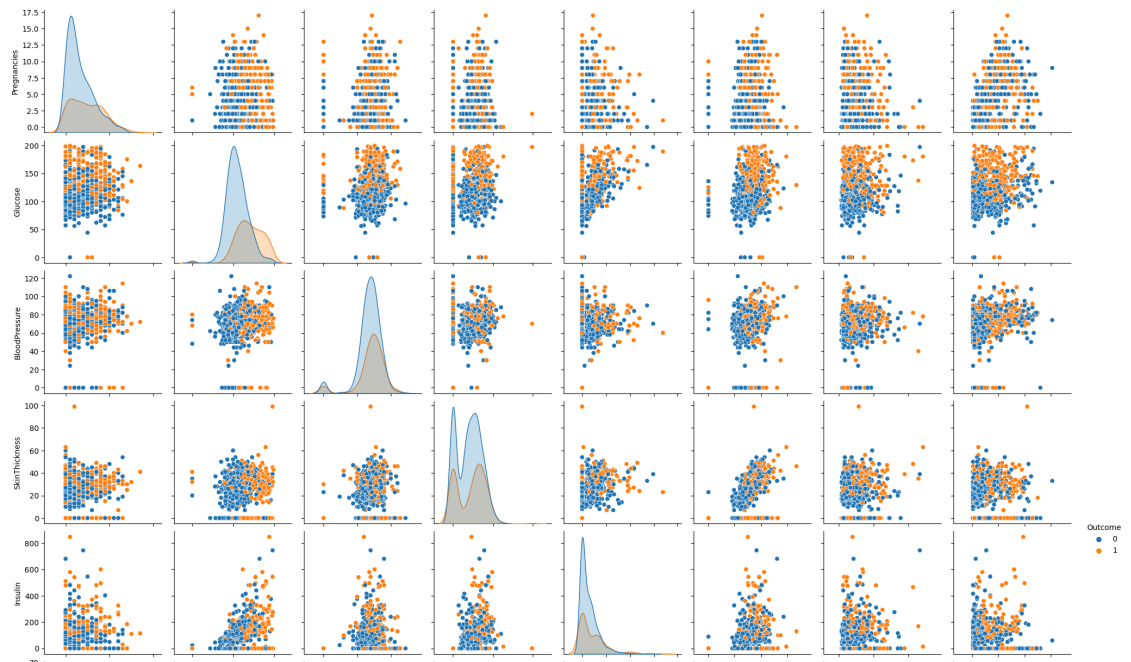
Out[6]:

```
Pregnancies      768
Glucose           768
BloodPressure     768
SkinThickness     768
Insulin           768
BMI               768
DiabetesPedigreeFunction  768
Age               768
Outcome           768
dtype: int64
```

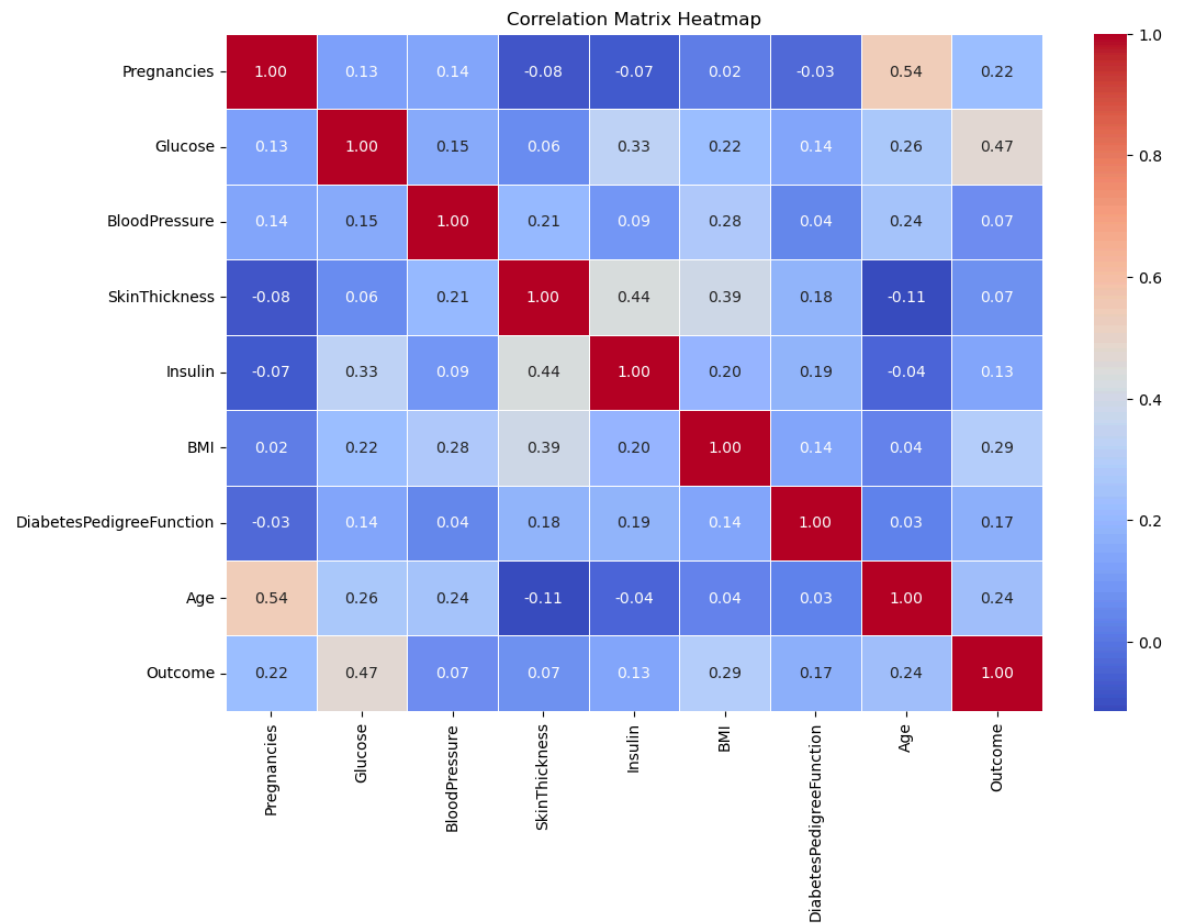
EDA

Understanding the dataset and its features is crucial. You'd need to analyze distributions, correlations, and outliers in the data to gain insights. For instance, examining the relationships between features like Glucose levels, BMI, and the outcome of diabetes could reveal significant patterns.

```
In [7]: sns.pairplot(df, hue='Outcome', diag_kind='kde')  
plt.show()
```



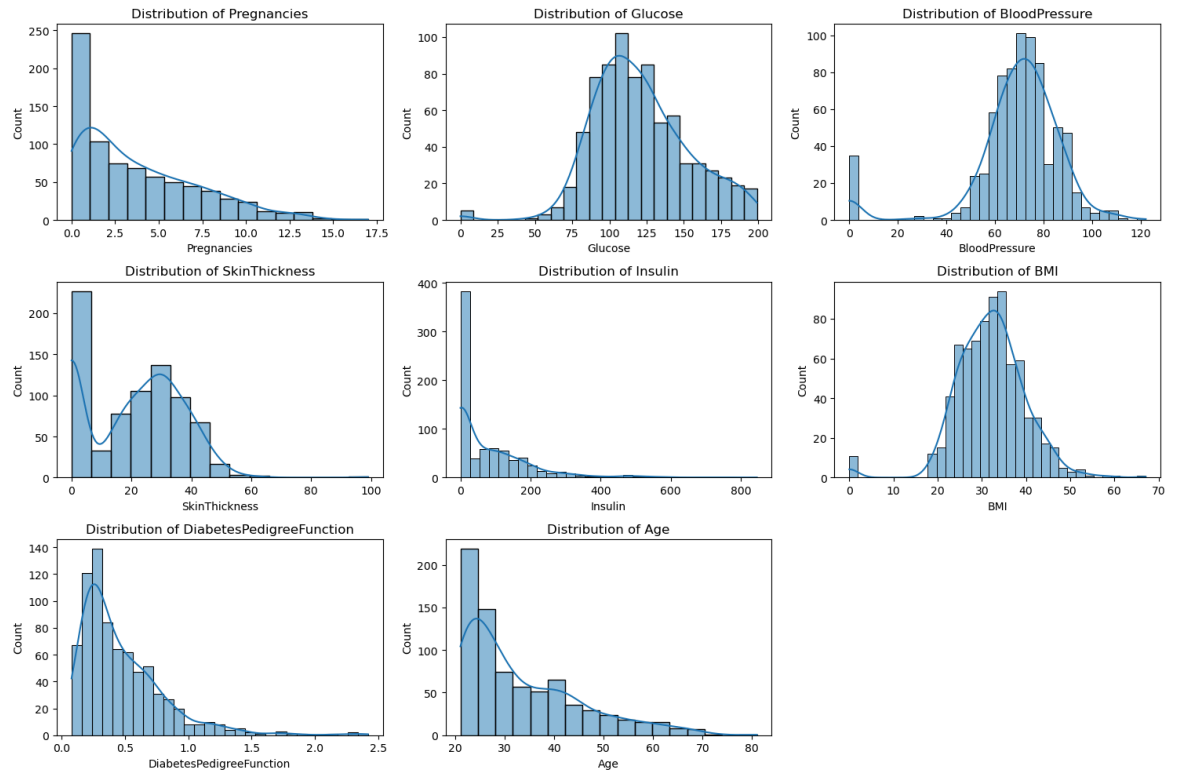
```
In [18]: ▶ plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title('Correlation Matrix Heatmap')
plt.show()
```



```
In [19]: ▶ numeric_features = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
```

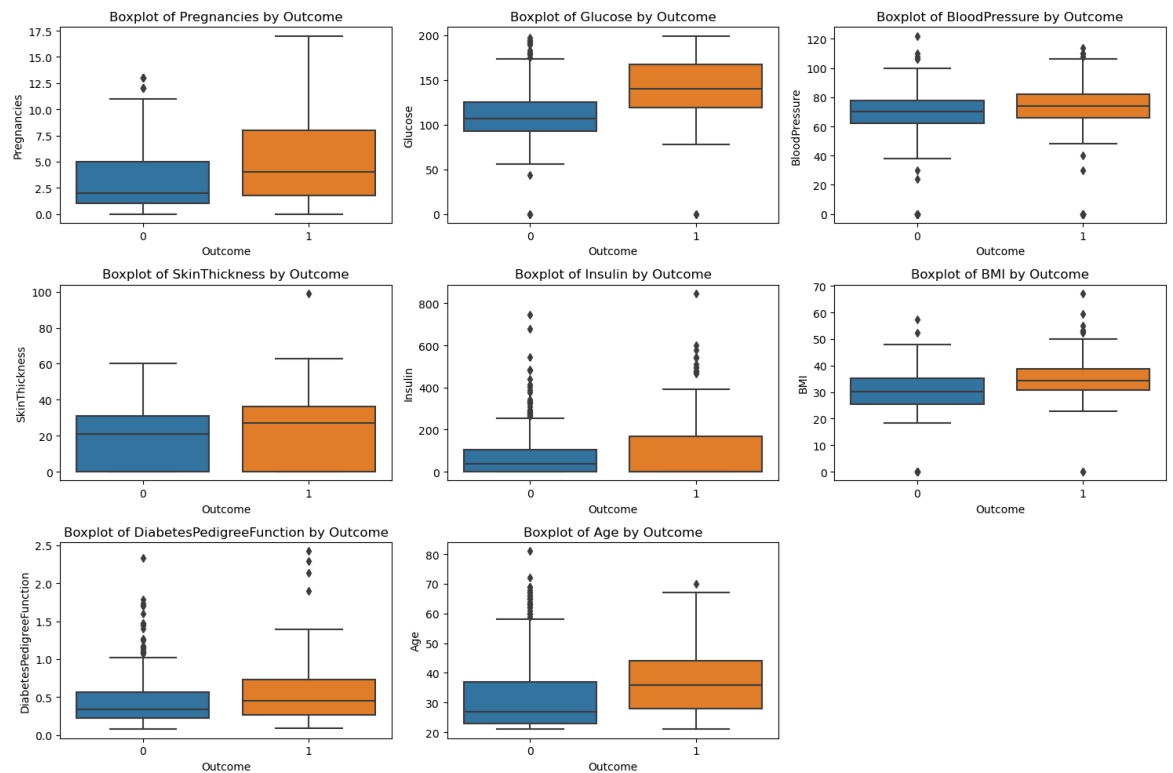
```
In [20]: ▶ plt.figure(figsize=(15, 10))
for i, feature in enumerate(numeric_features, 1):
    plt.subplot(3, 3, i)
    sns.histplot(df[feature], kde=True)
    plt.title(f'Distribution of {feature}')
    plt.xlabel(feature)

plt.tight_layout()
plt.show()
```



```
In [21]: ▶ plt.figure(figsize=(15, 10))
for i, feature in enumerate(numeric_features, 1):
    plt.subplot(3, 3, i)
    sns.boxplot(x='Outcome', y=feature, data=df)
    plt.title(f'Boxplot of {feature} by Outcome')
    plt.xlabel('Outcome')
    plt.ylabel(feature)

plt.tight_layout()
plt.show()
```



Data Modelling

The accuracy of the decision tree model on the test set is a critical factor in assessing its effectiveness. A high accuracy score indicates the model's ability to correctly classify individuals into diabetic or non-diabetic categories based on the provided features.

```
In [8]: ▶ # Split the data into features (X) and target variable (y)
X = df.drop('Outcome', axis=1)
y = df['Outcome']
```

```
In [9]: ▶ X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rand
```

```
In [10]: ▶ model = DecisionTreeClassifier(random_state=42)
```

```
In [11]: ▶ model.fit(X_train, y_train)
```

```
Out[11]: DecisionTreeClassifier(random_state=42)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
In [13]: ▶ y_pred = model.predict(X_test)
```

```
In [14]: ▶ accuracy = accuracy_score(y_test, y_pred)
          ▶ conf_matrix = confusion_matrix(y_test, y_pred)
```

```
In [15]: ▶ print(f'Accuracy: {accuracy}')
          ▶ print('Confusion Matrix:')
          ▶ print(conf_matrix)
```

```
Accuracy: 0.7467532467532467
Confusion Matrix:
[[75 24]
 [15 40]]
```

Decision trees offer interpretability, showcasing rules used to make predictions. Clear interpretation of these rules is essential in a healthcare setting. For example, identifying specific thresholds in features like Glucose levels or BMI that contribute significantly to predicting diabetes risk is valuable information for healthcare professionals.

In a healthcare setting, this model could be utilized for early identification or screening of individuals at risk of diabetes. However, caution should be exercised in implementing such models in real-world scenarios. Factors such as ethical considerations, model transparency, potential biases, and the need for human validation and expert intervention should be taken into account to ensure safe and reliable application in clinical practice.

```
In [17]: ▶ plt.figure(figsize=(12, 8))
plot_tree(model, filled=True, feature_names=X.columns.tolist(), class_names=[
plt.show()
```

